Bank IQ

A Machine Learning Intelligence Layer for Bank-Statement Analytics — transaction classification, customer segmentation and spend forecasting on a Regulatory Digital Twin

Rules decide. Models learn. Both shown side by side.

22MDC56 Machine Learning Laboratory · Project Work
Sandhru D. · Register No. _____ · M.Sc. Decision and Computing Sciences, V Semester
Guide: _____ · Coimbatore Institute of Technology · 2026–27

Problem identification

A statement records everything and explains nothing — and the rule-based classifier cannot learn

🏷️ Categorise

Which line is a charge, a refund, income? Today: regular expressions — exact on known formats, silent on the long tail.

👥 Segment

Which customers behave alike? Today: nothing — the engine works line by line.

📈 Forecast

What will next month cost? Today: nothing.

🚩 Notice

Which debit is unusual? Today: only what a rule names.

Problem statement

Given the transaction lines of a bank account (date, narration, debit, credit, balance), build, compare and deploy classical ML models that classify each line into ten categories more accurately than the existing regex classifier, predict next-month spend, segment customers by behaviour, flag unusual debits explainably, and present the results as business insights inside the existing app — with the rule engine still the only thing that files a claim.

Base: Vasool Raja (HackVerse 2.0, team Mad Angles) — parsers, digital twin, 19 RBI rules, Tamil/English app, 101 tests. Unchanged. Everything in bankiq/ is new.

Objectives — one per syllabus concept

All seven met, each against a baseline

1 1 Regression + CV

Next-month spend: Linear, Ridge, Lasso, Poly, DT, RF, GB — GroupKFold by customer; naive baseline

2 2 DT · NB · kNN

Transaction category (10 classes): + Logistic Regression; stratified 5-fold CV; regex baseline; per-class F1

3 3 K-Means vs Ward

Customer segments on 13 behavioural features — elbow, silhouette, dendrogram, ARI vs hidden type

4 4 PCA / SVD

PCA on profiles (variance, loadings, segment recovery); Truncated SVD on TF-IDF text

5 5 Ensembles

Bagging, Random Forest, AdaBoost, Gradient Boosting vs single tree; n-gram importances

6 6 LWR (from scratch)

Gaussian-kernel locally weighted regression of spend on income; bandwidth τ

7 7 End-to-end application

FastAPI endpoints + Intelligence dashboard in the existing Tamil/English app: KPIs, insights, category mix, monthly chart, segment, forecast, anomalies, model card

Literature survey — 16 references, one gap

Text representation → learners → ensembles → non-parametric → clustering → evaluation → financial anomalies

Area	Key references	What we use
Text features	Salton & Buckley 1988 (TF-IDF); Cavnar & Trenkle 1994 (char n-grams); Rennie et al. 2003 (Complement NB)	TF-IDF word 1–2 + char 3–5 grams; Complement NB
Single learners	Quinlan 1986 (trees); Cover & Hart 1967 (kNN)	Decision Tree, kNN with sweeps
Ensembles	Breiman 1996, 2001; Freund & Schapire 1997; Friedman 2001	Bagging, RF, AdaBoost, Gradient Boosting
Non-parametric	Cleveland 1979 (LOWESS)	LWR from scratch
Clustering + DR	MacQueen 1967; Ward 1963; Rousseeuw 1987; Jolliffe & Cadima 2016	K-Means, Ward, silhouette, PCA/SVD
Evaluation / finance	Kohavi 1995 (CV); Ahmed et al. 2016 (financial anomaly survey)	Stratified & grouped CV; robust z-score flags

Gap

Each paper studies one technique; none is applied to bank-statement data against the regex rule systems banks actually run; classification, segmentation, forecasting and anomaly flags are never built on one pipeline; nothing is deployed beside a consumer-protection rule engine; and no public labelled statement dataset exists. Bank IQ addresses all four.

System architecture

The rule engine and the intelligence layer share the parsers; only the rule engine files claims

1 Statement

CSV · PDF · passbook photo → vasool.parsers → lines

→

2 Features

clean narration → TF-IDF word + char n-grams + 9 numeric · customer-month panel (34) · customer profile (14)

→

3 Models

LogReg / NB / DT / kNN / RF / GB classifiers · 7 regressors · K-Means / Ward · PCA / SVD · LWR

→

4 Service

analyze(): categories (+ rule side by side) → mix → panel → segment → forecast → anomalies → insights

→

5 App

/api/ml/* + /api/accounts/{id}/intelligence · Intelligence screen (Tamil/English) · model card

Rule engine (unchanged)

classify (regex) → twin → 19 RBI rules → findings → evidence pack → guardian → case. The ML layer never creates a finding.

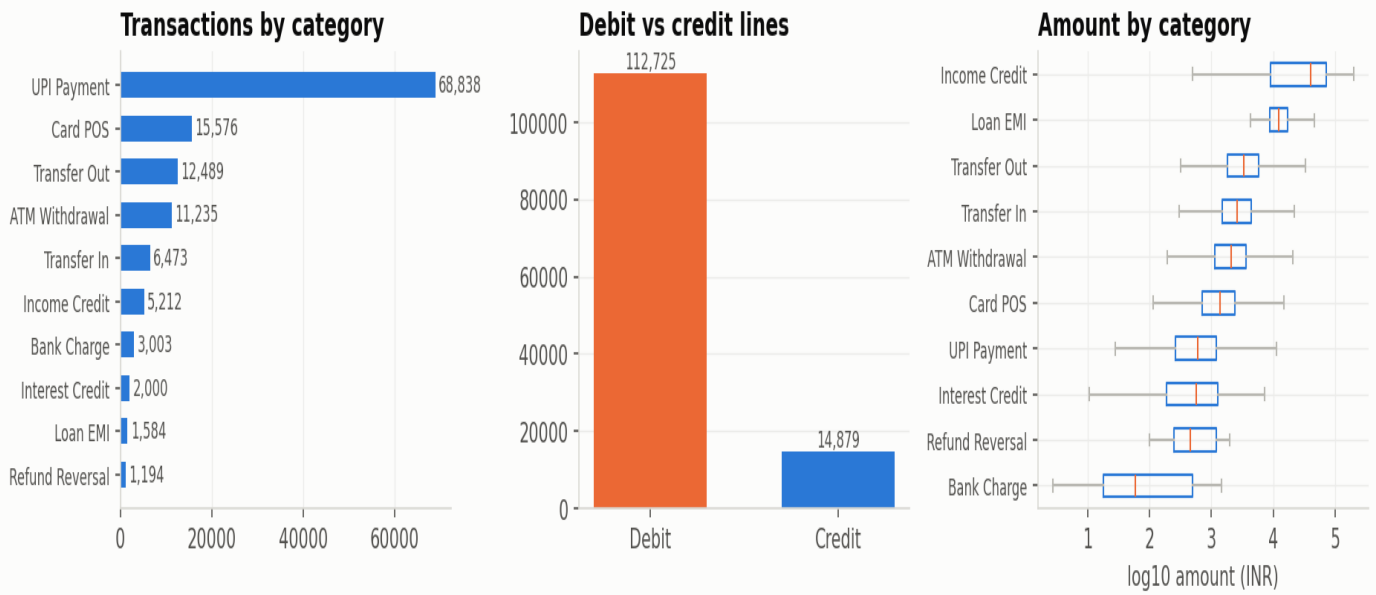
✓ Reproducible

make train → results/*.json, 15 figures, 6 joblib models (~25 min). Every table in the report, slides and model card is generated, not typed. 114 tests.

Dataset and preprocessing

1,27,604 labelled lines · 500 customers × 12 months · 10 categories · 5 hidden archetypes

Fig. 1 Transaction dataset: 127,604 lines · 500 customers · 12 months



- No public labelled statement data (personal data) → documented, deterministic generator; label = generating category, never a function of the narration
- 35% long-tail narrations + noise (truncation, codes) — where regexes fail
- Seasonality (festival ×1.25, school fees ×1.15) + one-off big-ticket months
- Cleaning: letter–digit split, mask ≥2-digit runs → NUM
- TF-IDF word 1–2 + char_wb 3–5 + 9 numeric → 2,789 sparse features; SVD-200 for kNN / boosting
- Customer-month panel (34 features, target = next spend); 14-feature profile

Exploratory data analysis

Spend is sticky and seasonal; UPI and ATM shares separate customer types; charges are rare but many-templated

Fig. 2 Time structure and customer mix

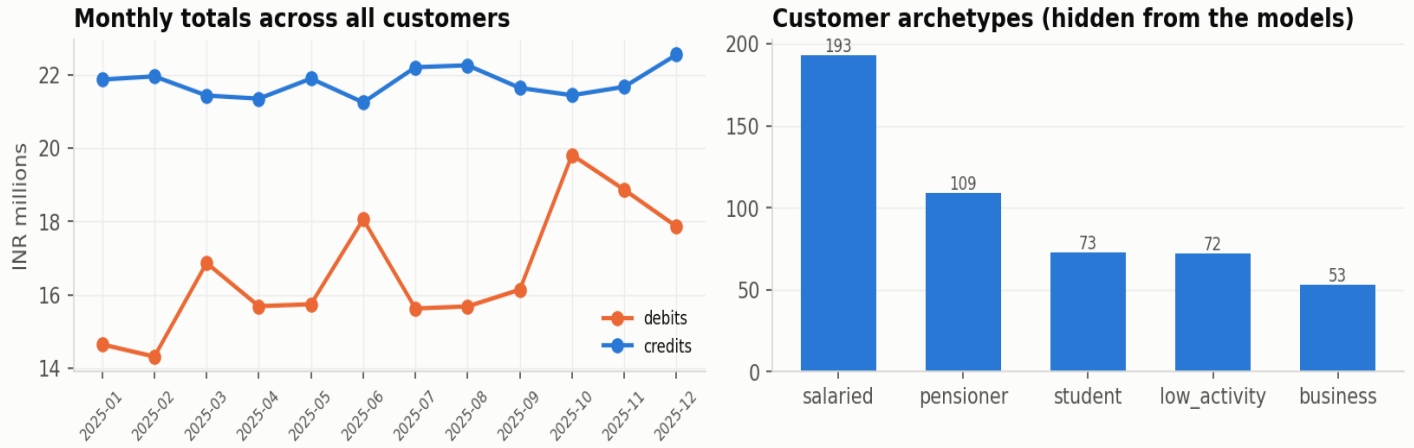
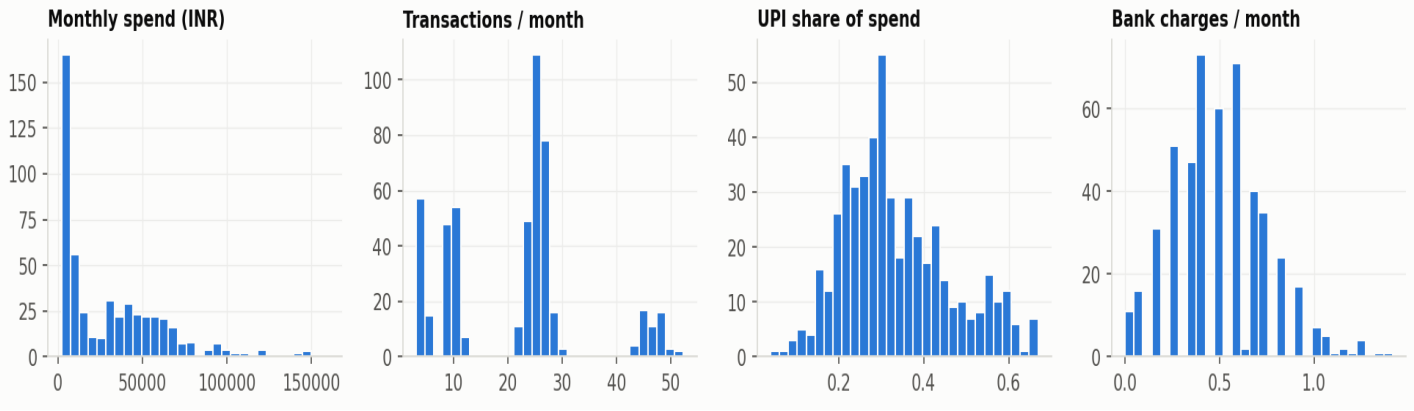


Fig. 3 Customer-level behaviour (500 customers)



- 88.3% of lines are debits; UPI is the most frequent category, bank charges and interest the rarest
- Amounts span 4 orders of magnitude → log amount, is_round, has_paise carry signal
- Monthly spend autocorrelated → naive forecast is a serious baseline
- Festival-season bump in every archetype → calendar features
- Students: UPI-heavy; pensioners: ATM-heavy; business: large transfers

Concept 2 · DT, Naive Bayes, kNN vs the regex baseline

Stratified 5-fold CV on 32,000 train lines · 8,000 held-out · 10 classes · macro-F1 drives selection

Model	CV acc	Test acc	Macro-F1
Rule-based (regex)	–	71.5%	0.794
Logistic Regression	99.9%	99.9%	0.997
Naive Bayes (Complement)	93.5%	93.4%	0.899
Decision Tree	97.0%	96.7%	0.972
kNN (k=7, SVD-200)	97.3%	97.4%	0.977

- Regex rules: 71.5% — perfect on known formats, blind on the long tail (lowest F1: Transfer out, Loan EMI)
- Logistic Regression 99.9% / macro-F1 0.997 — deployed
- Complement NB 93.4% in 0.4s (fastest, weakest) · tree 96.7% · kNN 97.4% on SVD-200 + cosine

Fig. 5 Transaction classification

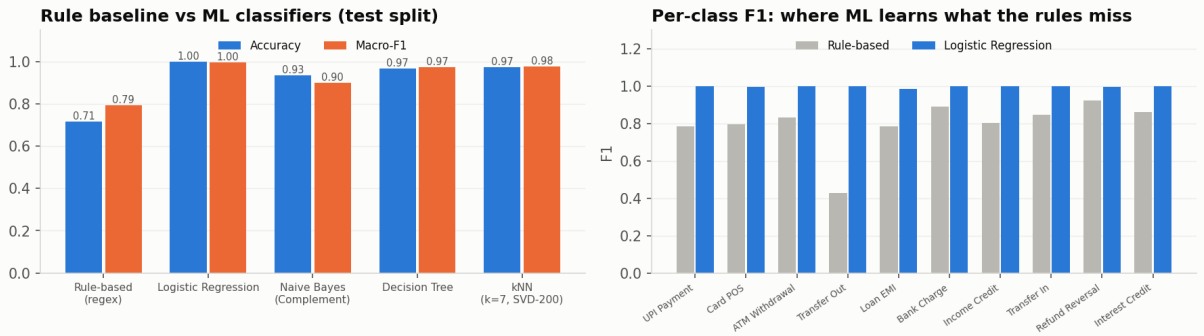
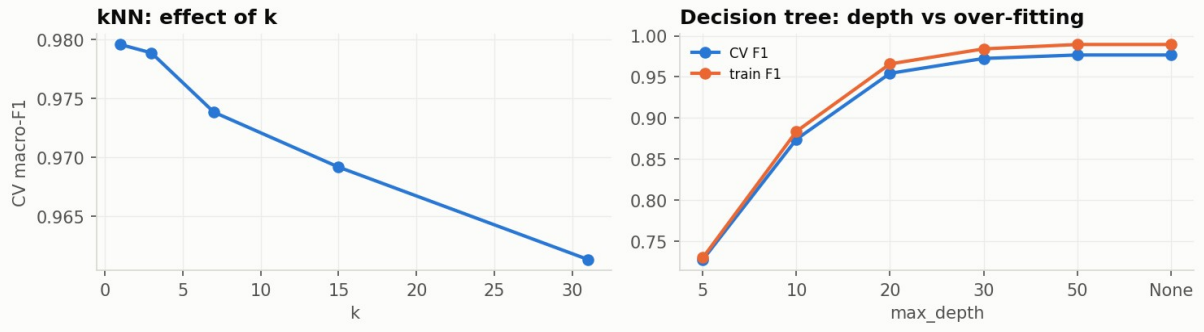


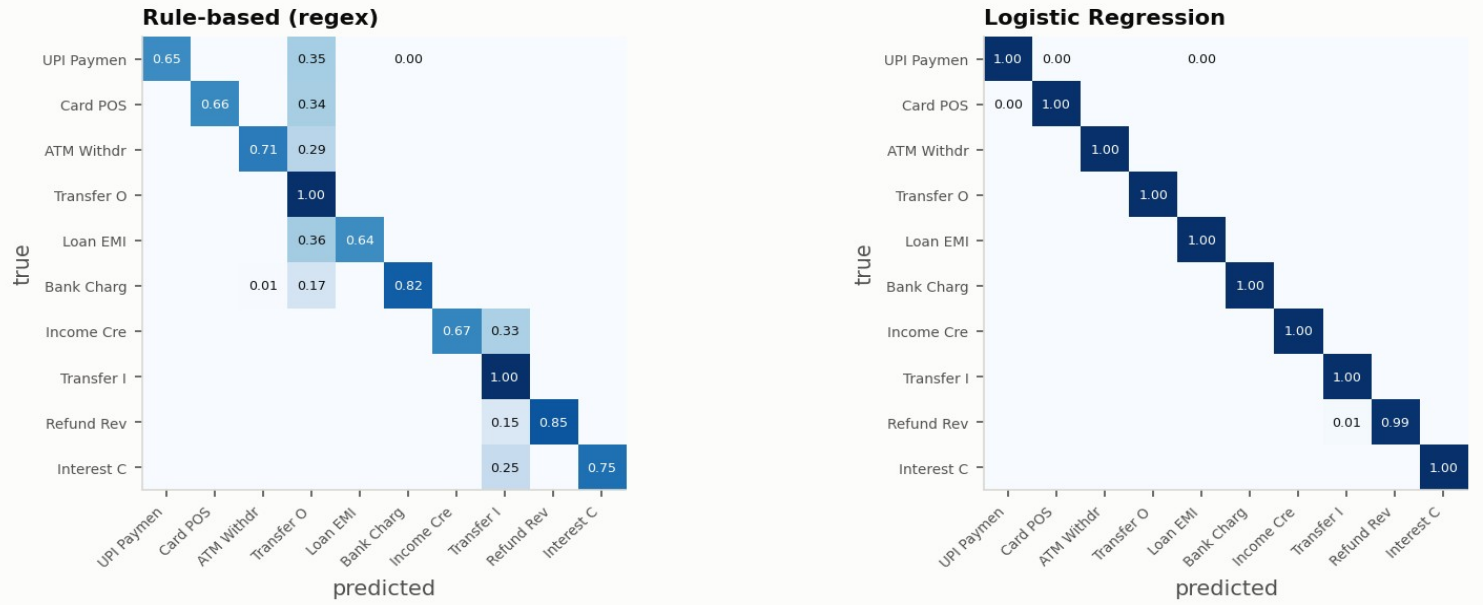
Fig. 7 Hyperparameter sweeps



Where the learned model gains

Row-normalised confusion: rules vs best model — the rules fail where they were never written (transfers, EMIs, merchant lines)

Fig. 6 Row-normalised confusion matrices



Category	Rule F1	ML F1	Gain
Transfer out	0.427	1.000	+0.573
UPI	0.785	0.999	+0.214
Loan EMI	0.783	0.985	+0.202
Card/POS	0.794	0.996	+0.202
Income	0.804	1.000	+0.196

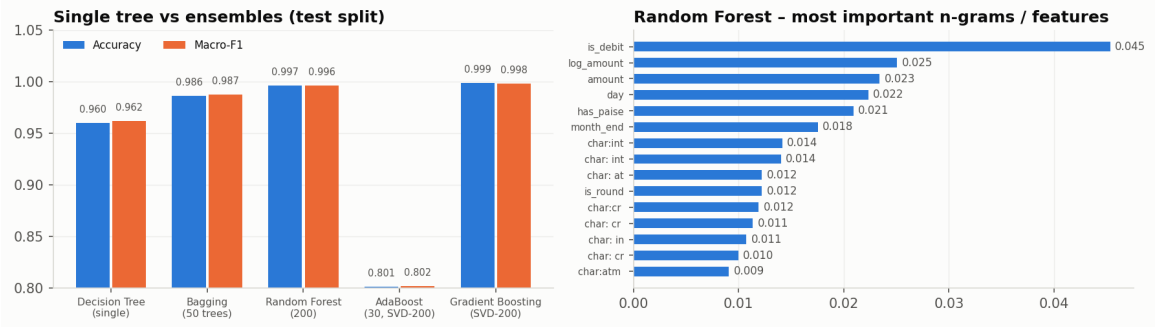
- Rules: weakest on transfers out (F1 0.427), EMIs, UPI/card merchant lines; strongest on charges (0.888) — what they were built for. Model: every class ≥ 0.985
- Rule-friendly sample statements: 99% agreement (validation). Long-tail sample (merchant-only, bank codes): 30 of 41 lines differ and the rules are the ones that fail — PAYROLL, BAJAJ FIN, SWIGGY CHENNAI, CSH WDL, QTRLY INT all become generic transfers
- Real-format check: 94 sample-statement lines → numeric block down-weighted to 0.1 (agreement with rules 83.0% → 98.9%); direction mask: a credit line can only be a credit category

Concept 5 · Ensemble learning

Bagging · Random Forest · AdaBoost · Gradient Boosting vs one tree · best: Gradient Boosting (SVD-200)

Model	CV acc	Test acc	Macro-F1	Fit s
Decision Tree (single)	96.1%	96.0%	0.962	5.6
Bagging (50 trees)	98.7%	98.6%	0.987	61
Random Forest (200)	99.6%	99.7%	0.996	32.6
AdaBoost (30, SVD-200)	84.4%	80.1%	0.802	143.3
Gradient Boosting (SVD-200)	99.6%	99.9%	0.998	57.4

Fig. 8 Ensemble learning for transaction classification



- Bagging: 96.0% → 98.6% by averaging tree variance
- Random Forest de-correlates trees too: 99.7%, macro-F1 0.996
- Boosting on SVD-200: 99.9%; AdaBoost stumps weakest (80.1%) and slow on 10 classes
- Forest's top n-grams are human-readable: is_debit, log_amount, amount, day

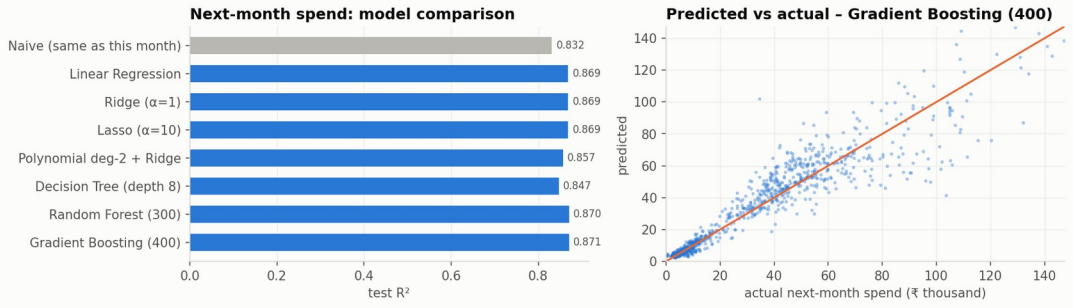
Concept 1 · Regression with cross-validation

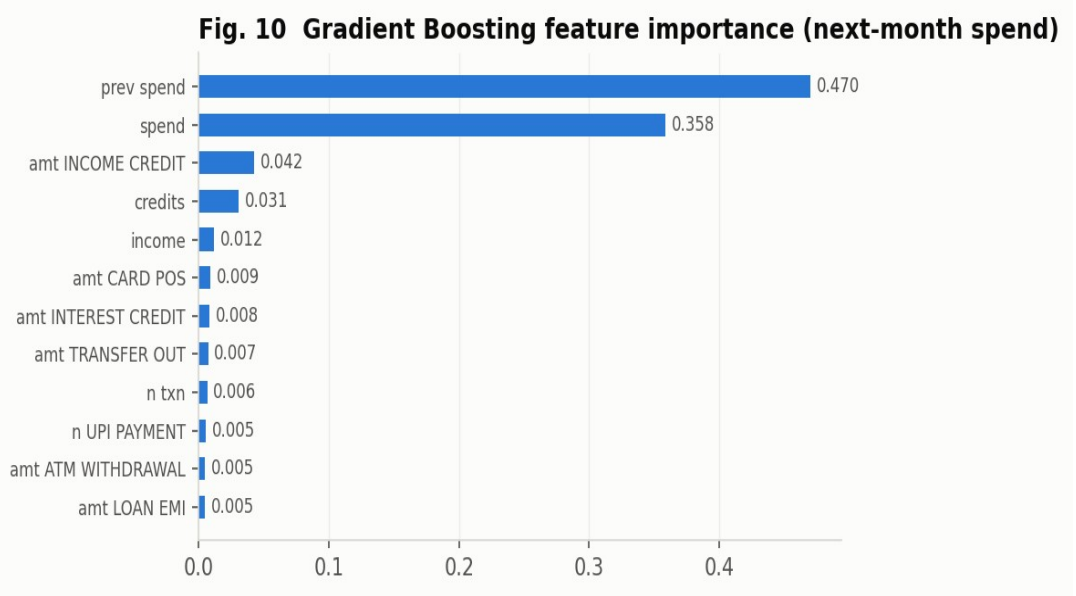
Next-month spend from this month's behaviour · GroupKFold(5) by customer · best: Gradient Boosting (400)

Model	CV R^2	Test R^2	RMSE	MAE
Naive (same as this month)	–	0.832	₹13,590	₹7,255
Linear Regression	0.863	0.869	₹11,990	₹6,653
Ridge (α=1)	0.863	0.869	₹11,990	₹6,652
Lasso (α=10)	0.863	0.869	₹11,993	₹6,636
Polynomial deg-2 + Ridge	0.852	0.857	₹12,529	₹6,654
Decision Tree (depth 8)	0.839	0.847	₹12,929	₹6,944
Random Forest (300)	0.875	0.870	₹11,915	₹6,313
Gradient Boosting (400)	0.875	0.871	₹11,881	₹6,105

- Naive "same as this month" already R^2 0.832 — spend is sticky
- Gradient Boosting (400): R^2 0.871, RMSE ₹11,881 — 13% below naive
- Ridge = Lasso = OLS: 4,400 rows support 34 coefficients; poly over-fits slightly
- Importance: previous + current spend ≈ 80%; one year of data → calendar effects barely learnable (future: multi-year)
- GroupKFold: a customer's months never split across folds — no identity leakage

Fig. 9 Regression with GroupKFold cross-validation





Concept 3 · K-Means vs hierarchical clustering

k = 5 (silhouette plateau 5–6, smallest kept) · K-Means vs Ward ARI 0.93 · recovers the hidden customer type: ARI 0.93

Fig. 11 Choosing the number of segments

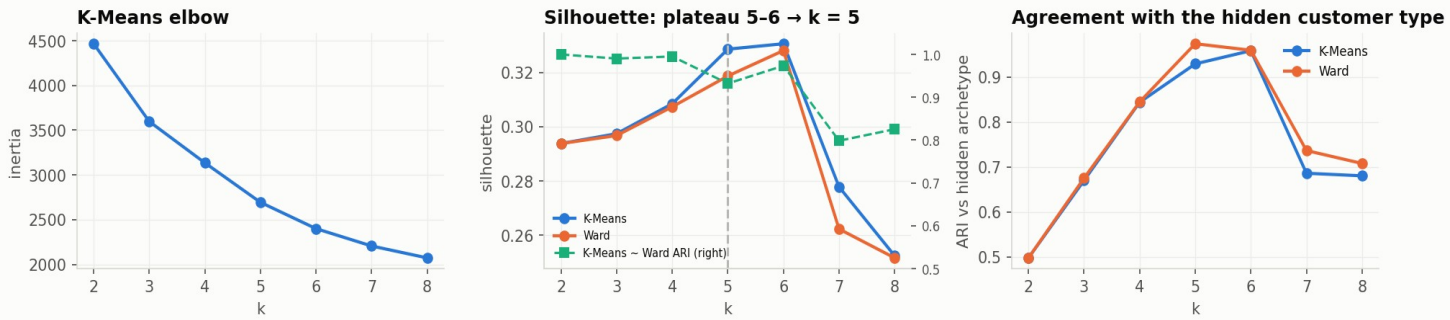
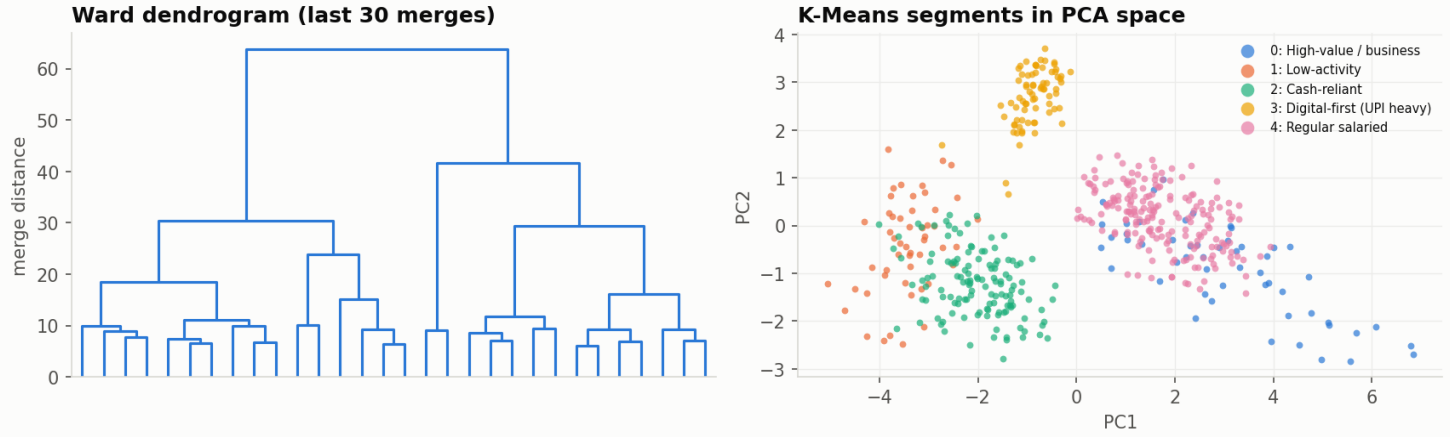


Fig. 12 Hierarchical structure and segment map



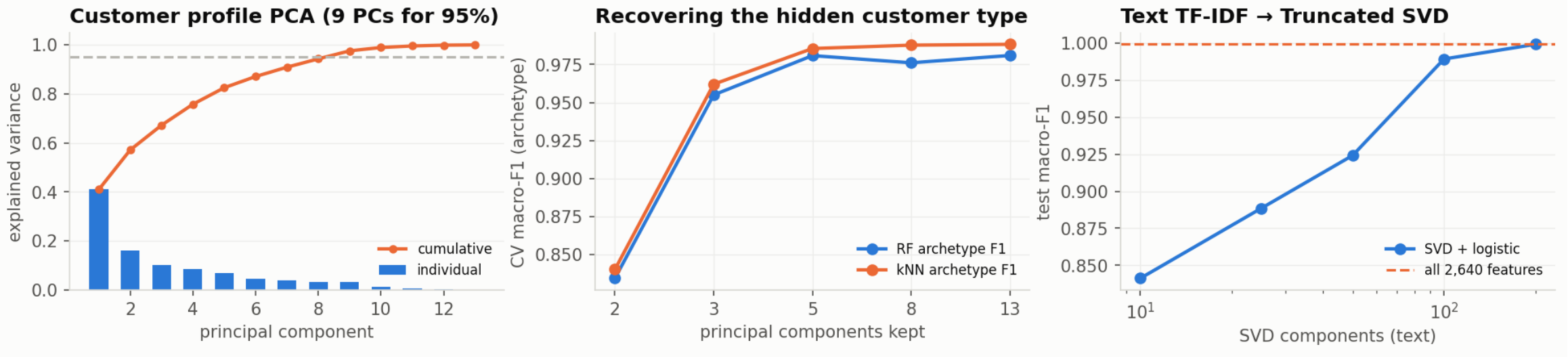
Segment	Size	Spend/mo	Type
High-value / business	53	₹95,051	business
Low-activity	51	₹3,020	low_activity
Cash-reliant	129	₹8,160	pensioner
Digital-first (UPI heavy)	74	₹5,979	student
Regular salaried	193	₹45,227	salaried

- 13 continuous behavioural features (median-month statistics), standardised; archetype used only to score
- has_emi left out: a standardised binary dominates Euclidean distance and split every segment
- Cash-reliant = pensioners (41% ATM) · Digital-first = students (56% UPI, no EMI) · Low-activity = most charges, dormancy risk

Concept 4 · Dimensionality reduction

PCA: 9 of 13 profile components for 95% variance — but 3 recover the segment · SVD on text

Fig. 14 Dimensionality reduction: PCA on profiles, SVD on narrations



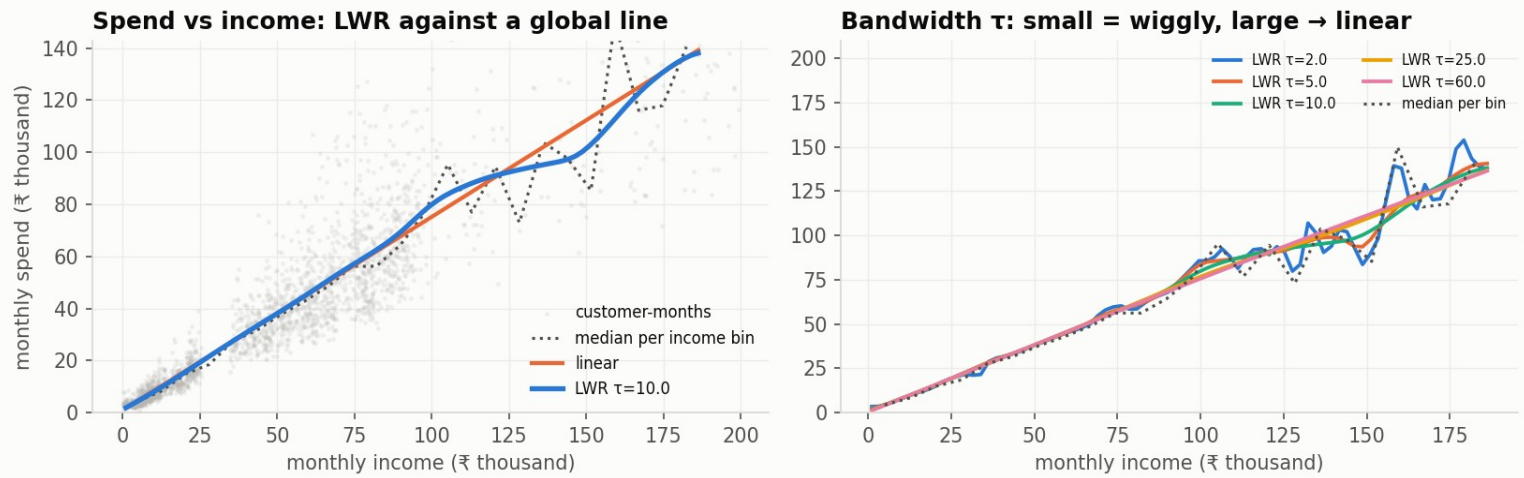
PCs kept	2	3	5	8	13
RF archetype F1	0.834	0.955	0.981	0.976	0.981
kNN archetype F1	0.840	0.962	0.986	0.988	0.988

- Profile features only mildly correlated → 9 PCs for 95% variance
- 3 PCs already give archetype F1 0.955 — the segment signal is 3-D
- Text: SVD-100 keeps 87.6% variance, logistic F1 0.989 vs 0.999 on all 2,789 features — speed for kNN/boosting at small cost

Concept 6 · Locally weighted regression (from scratch)

Gaussian kernel weights · weighted least squares at every query income · bandwidth τ sweep

Fig. 15 Locally weighted regression (Gaussian kernel, from scratch)

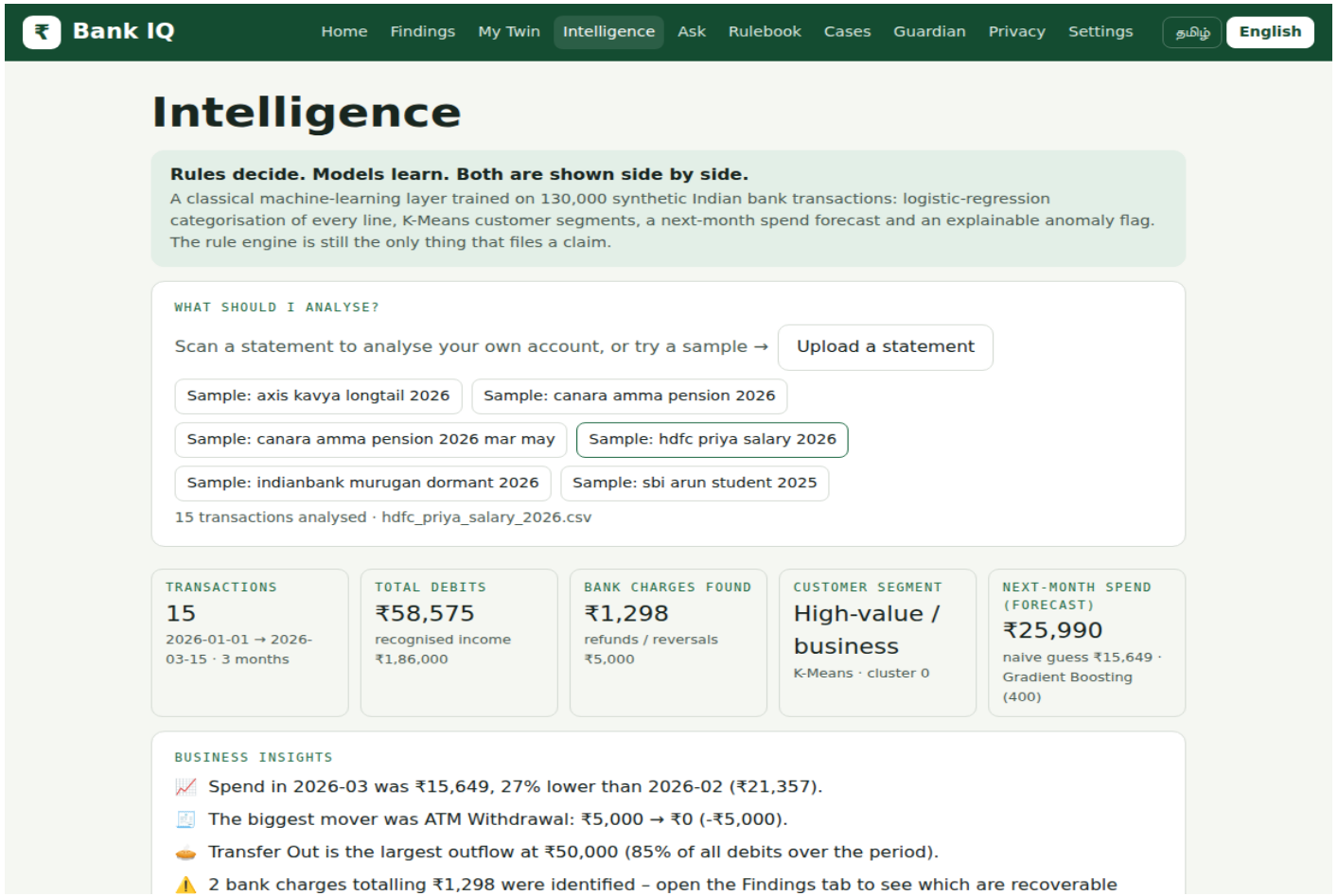


Model	Test RMSE
Linear	₹12,586
Polynomial deg-3	₹12,517
LWR τ=2.0	₹12,704
LWR τ=5.0	₹12,516
LWR τ=10.0	₹12,476
LWR τ=25.0	₹12,519
LWR τ=60.0	₹12,538

- $w_i = \exp(-(x_i - x_0)^2 / 2\tau^2)$; $\theta = (X^T W X)^{-1} X^T W y$; $\hat{y}_0 = [1, x_0]\cdot\theta$ — NumPy only
- Best LWR τ=10.0: RMSE ₹12,476 vs ₹12,586 for the global line
- Spend rises steeply with income at the low end and flattens — one line cannot bend
- Small τ chases noise in the sparse high-income tail; large τ → the global line (bias–variance)

Concept 7 · The Intelligence dashboard

FastAPI endpoints + a screen in the existing Tamil/English app · models inform, rules decide



- Source: my account · upload · samples
- KPIs: debits, income, charges, segment, forecast vs naive
- Insights engine: trend, biggest mover, largest outflow, charges → Findings, savings rate, segment, forecast, anomalies, rule disagreements
- Every line: ML category + confidence + rule category; 'show disagreements only'
- Unusual debits: robust z-score within category (log amount, 3 MADs)
- Model card: live tables from results/*.json + all 15 figures
- Endpoints: /api/ml/status · /results · /figures · /analyze · /analyze-json · /api/accounts/{id}/intelligence (session)

Result analysis and business insights

What the numbers mean for a customer, a bank and the rule engine

₹ Where the money goes

Every line now has a trusted category (rules: 71.5%; model: 99.9%). Charges still reach the rule engine: F1 0.888 → 1.000; the dashboard points them to Findings.

👥 Segments as actions

Cash-reliant pensioners: free-ATM rules. Low-activity: most charges, dormancy / reactivation charges. Digital-first students: failed-UPI reversals. Salaried: EMIs, forecast audience. High-value: widest forecast intervals.

📈 Expect next month

Forecast shown beside the naive guess with the model named; 13% lower error on test; gains concentrate in seasonal months.

🚩 Unusual debits

Category-aware: ₹15,000 is normal for a transfer, abnormal for UPI. Insight says '4× the typical ATM withdrawal'. Hints, never claims.

! What not to trust

99% on synthetic data = learned the generator's language; trust the per-class ordering and the sample-statement disagreements. Forecasts need ≥3 months. Nothing here is a claim.

Deployment, testing and demo

One command to reproduce every number · one command to run the app

▶ Run

make train → results/, models/ (~25 min)
uvicorn api.main:app → :8000 → Intelligence
docker compose up --build (trains at build)

✓ Tested

101 Vasool Raja tests + 13 new: generator determinism, masking, featurizer, panel/profile, rule map, LWR exactness, service accuracy > 90%, endpoints, 415 on bad upload, 401/404 without session.

⌥ Repository

bankiq/ (data, features, rulemap, experiments, service, api) · notebooks/01–07 · results/ · models/ · tests/ · docs/ (Phase I, literature, report, slides, viva)

3-minute demo

- Intelligence → Sample: hdfc priya salary → KPIs, insights, category mix (model and rules agree on every line)
- Sample: axis kavya longtail → 'show disagreements only': 30 of 41 lines — the regexes see generic transfers, the model sees income, EMI, UPI, ATM, refund, interest, charge
- Canara pension sample → Cash-reliant, five charges, ₹50,000 college fee flagged
- Model card → Table live from results JSON → Fig 5, Fig 11, Fig 15
- Findings tab → the rule engine's claims on the same statement

Conclusion and future scope

- All seven syllabus techniques applied to one real product, each against a baseline, under cross-validation that respects the data (stratified; grouped by customer)
- Transaction classifier: 71.5% (regex) → 99.9% (logistic) / 99.7% (forest); biggest gains on transfers, EMIs and merchant lines
- Forecast 13% below naive; K-Means and Ward agree on 5 segments that recover the hidden types (ARI 0.93); PCA shows a 3-D segment signal; LWR exposes the bend in spend vs income
- Deployed beside an untouched rule engine; reproducible; 114 tests

Future scope

- Real consented statements (Account Aggregator) + active learning on model–rule disagreements
- Multi-year histories; prediction intervals
- Confirmed findings fed back as training labels
- Tamil / Hindi tokens; OCR-specific character models
- Isolation Forest once labelled anomalies exist

github.com/<your-user>/bank-iq · Thank you — questions?